electronics

MDPI

Article

Study on a User Preference Conversational Recommender Based on a Knowledge Graph

Ganglong Duan, Shanshan Xie * and Yutong Du

School of Economics and Management, Xi'an University of Technology, Xi'an 710054, China; gl-duan@xaut.edu.cn (G.D.); 2230520059@stu.xaut.edu.cn (Y.D.)
* Correspondence: 2230520062@stu.xaut.edu.cn; Tel.: +86-18192677527

Abstract: In the era of information explosion, as a form of personalized recommendation, dialogue recommendation systems provide users with personalized recommendation services through natural language interaction. However, in the face of complex user preferences, the traditional dialogue recommendation system has the problem of a poor recommendation effect. To solve these problems, this paper proposes a user preference dialogue recommendation algorithm (KGCR) based on a knowledge graph, which aims to enhance the understanding of user preferences through the semantic information of the knowledge graph and improve the relevance and accuracy of recommendations. This paper proposes a personalized conversation recommendation algorithm framework for user preference modeling. The framework uses a bilinear model attention mechanism and self-attention hierarchical coding structure to model user preferences to rank and recommend candidate items. By introducing rich user-related information, the recommendation results are not only more in line with users' individual preferences but also have better diversity, effectively reducing the negative impact of information cocoons and other phenomena. At the same time, the experimental results on the open dataset prove the effectiveness and accuracy of the proposed model in the personalized conversation recommendation task.

Keywords: dialogue recommendation; knowledge graph; user preference; bilinear model

Academic Editor: Arkaitz Zubiaga

Received: 7 January 2025
Revised: 28 January 2025
Accepted: 3 February 2025
Published: 6 February 2025

Citation: Duan, G.; Xie, S.; Du, Y. Study on a User Preference Conversational Recommender Based on a Knowledge Graph. *Electronics* **2025**, *14*, 632. https://doi.org/10.3390/electronics14030632

1. Introduction

As a technical means, the recommendation system has been widely used in social networking, e-commerce, news, and other fields in recent years, and its commercial potential has gradually emerged. In the traditional recommendation system, the recommendation task is mainly realized based on the historical interaction record between the user and the item. For example, the e-commerce website will recommend the product [1] according to the user's clicks and purchase records. However, this method that relies on implicit feedback makes it difficult to accurately mine the real preferences of users, which limits the performance of the recommendation system. To solve this problem, the dialogue recommendation system has emerged. Such systems conduct multiple rounds of conversations with users through natural language, gradually and deeply understanding their personalized preferences and thus providing high-quality recommendation results [2]. Compared with the traditional recommendation system, the dialogue recommendation system can obtain timely and explicit feedback in the user interaction, more accurately reflect the real needs of users, and create a greater space for the performance improvement of the recommendation system. In the dialogue recommendation system, the design usually includes two parts: a recommendation module and a dialogue module. The conversation module is responsible

for understanding the user's intentions, providing information about user preferences to the recommendation module, and generating responses based on user feedback. The recommendation module searches for matches according to the information provided by the dialogue module in the item database to complete the recommendation task with higher quality. For example, in a movie recommendation scenario, the conversation recommendation system focuses on the user's favorite movie genre and provides a recommended movie in the response. The entire workflow ends when the user accepts suggestions from the system.

In this process, the system needs to complete both dialogue and recommendation tasks at the same time, so ensuring the effective connection and collaborative operation of the two modules is crucial to building an efficient dialogue recommendation system. Table 1 is a simplified example of the dialogue recommendation system interaction. The dialogue between the user (U) and the system (S) shows an example of the interaction between the dialogue recommendation system and the user in the movie recommendation scenario, reflecting the synergistic effect of the system in completing these two types of tasks.

Table 1. Dialogue recommendation system interaction example.

Character	Dialogue Content
U	Hi, I'm looking for some movies to watch.
S	Hello! I can recommend movies for you. What kind of movies do you prefer?
U	I like science fiction movies, especially those about space.
S	Okay, have you seen *Interstellar*?
U	Yes, I loved that movie.
S	Then you might like *Guardians of the Galaxy*. It's also a space science fiction film with a lot of humor.
U	Sounds good. I'll check it out. Any other recommendations?
S	Of course, you can also try "Alien Awakening," a sci-fi thriller about alien life.
U	Ok, thanks for your recommendation!
S	You're welcome. I hope you enjoy the movies. Please don't hesitate to let me know if you need any more help.

In recent years, the dialogue recommendation system has gradually become a research hotspot, and researchers have proposed a variety of algorithms to set different problems to meet different needs (ReDial [3]). The sentiment analysis module was introduced to more accurately obtain user preferences for candidate items; Shuokai Li and others [4] at the Institute of Computing Technology of the Chinese Academy of Sciences proposed the UCCR model and innovatively considered the integration of user historical sessions and similar user information to provide a more comprehensive understanding of users. UCCR models user interests more accurately in CRS while considering their entity, semantic, and consumption preferences (estimation–action–reflection (EAR) [5]). In the recommendation system phase, the system learns a dialogue strategy and decides whether to ask questions about the properties of the item or to recommend the item. This decision is based on the outcome and the conversation history. The goal of the action phase is to dynamically adapt the conversation to better understand user needs and to ask questions or provide recommendations when appropriate. These works improve the performance of the dialogue recommendation system to some extent, but there are limitations. First, existing studies have only modeled the current conversation, and to provide recommended items in a short time, the rounds need to be controlled within a certain range. This restriction on the depth of interaction leads to the system's one-sided understanding of users, and it is difficult to dig deep into users' interests and preferences for candidates. Secondly, users' preferences are extensive and multifaceted, whereas a single dialogue only reflects one aspect of users' preferences, which leads to the trend of homogenization of recommendation results, which

can easily cause bad phenomena such as an "information cocoon room". An example of a movie recommendation obtained by ignoring user history information is shown in Figure 1.

Figure 1. Example of a movie recommendation by ignoring the user history information.

To solve these problems, we can consider the modeling based on the user's historical dialogue data. In this way, the system can obtain more sufficient information from the historical dialogue data without increasing the current dialogue rounds to gain a deeper understanding of the user. At the same time, each user conversation may have a different theme, showing the user's interests and preferences in different aspects. Using this information can alleviate the lack of current dialogue information and improve the diversity of recommendation results. However, challenges still arise when modeling user history conversation data. First, not all historical dialogue transcripts are valuable for the current recommendation task. Second, user history conversation data are rich and redundant, and indiscriminate utilization may result in recommended results being irrelevant to the current task. Therefore, it is necessary to identify historical conversation data for information useful to the current task and to weigh the importance of historical conversation data against current conversation data.

To address these challenges, the KGCR (knowledge graph-based user preference conversational recommender) model is proposed. The model uses the attention mechanism to give weight to the items that interact with users in a historical conversation, and the greater the weight, the higher the fit to the current task to identify the most valuable information. At the same time, regarding the "target dialogue is given priority to, historical data is complementary" principle, based on the attention mechanism design hierarchy since the attention coder structure, first fusion historical dialogue data interact with the user; obtain the history-related user vector, with the entity in the current dialogue; and obtain the final user vector. This hierarchical coding not only considers the different interest preferences that users show in multiple conversations but also focuses on their needs in the current dialogue, realizing the fine modeling of user preferences. The experimental results show that KGCR improves its performance on both the recommendation and

the dialogue tasks relative to the baseline model, demonstrating the effectiveness of the present approach.

2. Related Work

2.1. Recommendation Algorithm Based on the Knowledge Graph

With the rapid development of the Internet, recommendation systems have become an important part of various online platforms. However, traditional recommender systems face some challenges, such as data sparsity and cold-start problems. To solve these problems, researchers began to explore the recommendation algorithms based on the knowledge graph to use the rich structured information in the knowledge graph to enhance the performance of the recommendation system. In terms of the construction and application of the knowledge graph, Nickel et al. [6] proposed a complex embedding model called ComplEx, which can effectively use the multi-relational data in the knowledge graph for recommendation. Moreover, Dong et al. [7] proposed a recommendation system based on the knowledge graph, which improves the accuracy and interpretability of the recommendation system by utilizing the entity and relational information in the knowledge graph. Graph neural networks (GNNs) have advantages in processing graph structural data, so they are also applied in recommendation systems. Hamilton et al. [8] proposed a model called GraphSAGE, which can generate embedding representations of nodes by sampling the information of neighbor nodes, and this method can be applied to the knowledge graph in the recommendation system. Furthermore, Wang et al. [9] proposed a model called KGAT, which combines knowledge graph and attention mechanisms to take into account both user–project interactions and higher-order relationships in the knowledge graph. However, the knowledge graph-based recommendation algorithms still face some challenges, such as the incompleteness of the knowledge graph and the scalability of the recommendation systems. To address these issues, future research directions include the development of more effective models to handle large-scale knowledge graphs and the improvement of the real-time and personalized level of recommendation systems.

2.2. Dialogue Recommendation System

A dialogue recommendation system is a system that can explore user dynamic preferences through multiple rounds of real-time dialogue and make recommendations accordingly. It combines the techniques of natural language processing and recommendation systems and is designed to better understand user preferences through multiple rounds of conversation. The architecture of a system usually includes the user interface module for communicating with users, the recommendation engine responsible for generating the recommended content, and the dialogue policy module [10] for managing the interactive logic and task flow of the entire system. The traditional recommendation system mainly relies on the user's historical data to infer the preferences, while the conversational recommendation system obtains the feedback through the real-time interaction with the user, which enables the system to adapt more dynamically and flexibly to the change of the user's preferences. This interactivity enables the conversational recommendation system to provide more personalized and accurate recommendation results.

In recent years, the field of conversation recommendation systems has gained widespread attention thanks to improved natural language processing technologies, such as the popularity of voice assistants and chatbots. In addition, the application of advanced technologies such as reinforcement learning and knowledge graphing in recommendation strategies also provides support for the development of dialogue recommendation systems [2]. The combination of these technologies makes conversational recommendation systems more effective in understanding and responding to user needs. Kun Zhou et al. [11]

proposed the KG-based semantic fusion approach (KGSF), which opened the semantic gap of different types of information in the dialogue through the semantic fusion technology of mutual information maximization, and the downstream model was designed to improve the effect of CRS. Zhu Lixi et al. [12] proposed a dialogue recommendation model NCPR that can effectively make use of negative feedback and make full use of the negative feedback of attributes and granularity of items given in the interaction process to dynamically correct the user's preference representation. Li Xian et al. [13] proposed a novel architecture, the long-term short-term feedback architecture, to connect these two basic components in CRS. Specifically, recommendations predict long-term recommendation goals based on the conversation context and user history. Driven by the target recommendation, the conversation model predicts the next topic or attribute to verify that the user preference matches the target. The research of dialogue recommendation systems involves a variety of methods, such as attribute-based dialogue recommendation, noise reduction collaboration, knowledge enhancement, semantic fusion, topic guidance, etc. These methods are all designed to better understand users' interests and preferences through conversations to provide more accurate recommendations. Kun Zhou et al. [14] proposed an open-source CRS toolkit, CRSLab, which provides a unified and scalable framework with highly decoupled modules to develop CRS. The UCCR model proposed by Li Shuo [4] innovatively considers the integration of user history sessions and similar user information to provide a more comprehensive understanding of users in CRS. Despite some progress in the research of dialogue recommendation systems, there are also some challenges: how to effectively evaluate the performance of the system, how to deal with the multi-intention problems of users, and how to improve the dynamic adaptability of the system need to be further explored and solved by researchers.

The field of interactive recommendation systems has witnessed significant advancements through the integration of various methodologies. While artificial intelligence (AI) has emerged as a dominant force, it is crucial to acknowledge the valuable contributions of traditional algorithms and techniques. Fuzzy logic allows for the representation and processing of uncertain and imprecise information, enabling the system to make more nuanced and flexible recommendations. Similarly, the exploration of Markov processes in recommendation systems, as seen in various Chinese research papers, offers valuable insights into user behavior modeling and prediction. There is the potential for knowledge graph-based approaches to enhance recommendation accuracy and interpretability. By leveraging structured information from knowledge graphs, these systems can capture complex relationships between entities and provide more relevant and personalized recommendations. Furthermore, the fusion of matrix factorization and deep learning, as exemplified by the NeuMF model, showcases the potential of combining traditional and AI-based techniques. This hybrid approach allows for the effective learning of user and item features, leading to improved recommendation performance. In conclusion, by referencing both Chinese and foreign cases, we gain a comprehensive understanding of the diverse methodologies and techniques employed in interactive recommendation systems. This cross-referencing enables us to identify the strengths and limitations of different approaches and facilitates the development of more innovative and effective recommendation models.

Traditional recommendation systems have long been dependent on precise user behavior data to generate accurate recommendations. However, the advent of fuzzy logic has introduced a novel perspective to the field of recommendation algorithms. Specifically, Karthik et al. [15] proposed an innovative fuzzy logic-based recommendation system that dynamically predicts relevant products in response to evolving user interests in online shopping scenarios. This system calculates sentiment scores using a unique algorithm and employs fuzzy recommendation logic combined with ontology matching to achieve

high-precision recommendations. The hybrid approach not only demonstrates the potential of fuzzy logic in capturing nuanced user preferences but also significantly improves the accuracy of recommendations. Building on this foundation, Aliberti et al. [16] introduced the concept of fuzzy user signatures, a technique that efficiently represents user interests and preferences through the application of fuzzy logic. This method enables accurate similarity calculations between users, even when data are limited. The practicality and superior performance of fuzzy user signatures are exemplified through their application in a movie recommendation system, highlighting the method's potential for delivering personalized recommendations across various domains.

To further enhance the capture of user preferences and predict future behavior, Rendle et al. [17] proposed a hybrid recommendation system that integrates matrix factorization (MF) and Markov chain Q (MC). This system employs a pairwise interaction model to effectively handle limited data and utilizes a Bayesian personalized ranking framework for parameter learning. The hybrid approach showcases the effectiveness of combining MF and MC in capturing user preferences and predicting future behavior, thereby improving the overall recommendation performance. In a similar vein, Yang et al. [18] introduced U2CMS, a hybrid recommendation system that integrates collaborative and content-based similarity models with Markov chains for sequential recommendations. This system employs a higher-order Markov chain to model sequential patterns and leverages textual content information to provide accurate and relevant recommendations. By effectively capturing both user preferences and temporal dynamics, this approach leads to improved recommendation accuracy and enhanced user satisfaction. In conclusion, these studies collectively illustrate the evolution and effectiveness of various recommendation algorithms. They highlight the potential of integrating diverse methodologies to enhance recommendation systems. From the application of fuzzy logic and ontology matching to the development of fuzzy user signatures, and from hybrid systems combining MF and MC to sequential recommendations with Markov chains, these studies provide valuable insights and directions for building smarter and more personalized recommendation systems.

3. The Proposed Model: KGCR

To effectively complete the personalized dialogue recommendation task, this paper proposes a new algorithmic framework—KGCR (knowledge graph-based user preference conversational recommender). A core innovation of the framework is that it is the integration of users' historical dialogue data as contextual information into the user modeling process. In this way, KGCR can gain a more comprehensive understanding of user interests and preferences.

Figure 2 details the structure of the recommended modules in the KGCR model. In this structure, we can see two main components: a baseline model and a history-based modeling module. The baseline model is formed based on the knowledge graph, where part of it is located on the right side of the figure, introducing the knowledge graph as an environment from the perspective of improving data quality in the case of minimizing user burden, designing interaction tasks to actively obtain user online feedback, thus being based on high-quality and targeted user preference feedback to achieve accurate and efficient dialogue recommendation. Another core module of KGCR, which is located on the left side of the graph, is used to generate a history-related user vector representation through historical conversation data. These representations not only enrich the expression power of the user vectors but can also more accurately match the items in the item database, thus improving the quality of the recommendations.

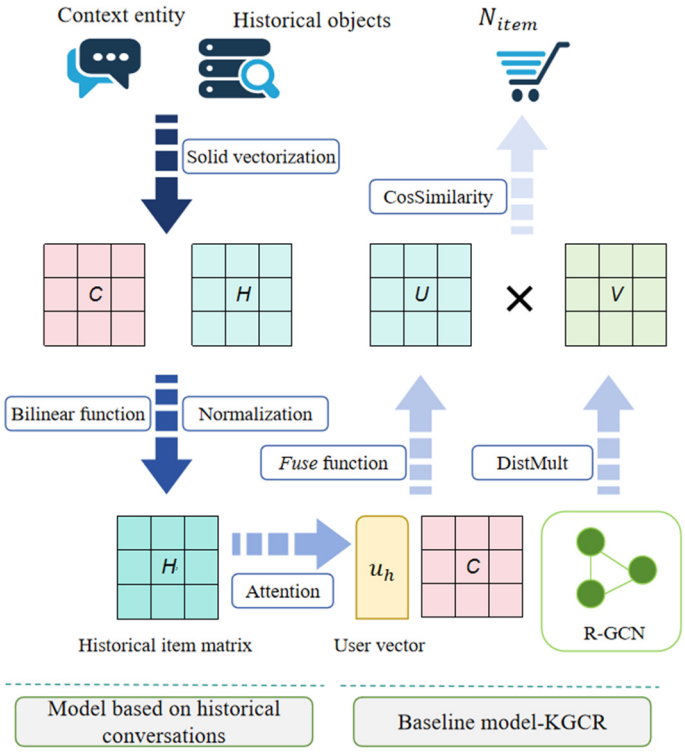


Figure 2. The proposed KGCR model consists of two components: a history-based modeling module and a baseline model.

3.1. Constructing the Entity Vector Based on the Knowledge Graph

The knowledge graph was encoded using R-GCN [19] to obtain a low-dimensional vector representation of the entities. The entities in the target dialogue and the historical dialogue are extracted to form the context entity set and the historical item set, and the corresponding entity vector matrix is obtained. From the perspective of data enhancement, this method models the environment information through the knowledge graph, enhances the key information in the atlas, designs effective interaction strategies, and realizes the accurate and efficient active interaction recommendation algorithm.

3.1.1. Building a Knowledge Graph

The user history interaction data are built into the user-object interaction graph, and the external knowledge graph is integrated to enrich the context information of the interaction environment. To construct the knowledge graph, we first integrate user interaction data into a user-object interaction graph. This graph captures the relationships between users and items based on historical interactions. For example, if a user has watched a movie, a corresponding edge is created between the user node and the movie node. Additionally, we integrate an external knowledge graph to enrich the context information of the interaction environment. This external knowledge graph contains entities such as movies, actors, directors, genres, and their relationships.

3.1.2. Enhancement of the Knowledge Graph

One of the significant challenges in using knowledge graphs for recommendation systems is the presence of missing or incomplete data. Incomplete knowledge graphs can lead to gaps in the information available for recommendation, potentially reducing the accuracy and relevance of the recommendations. To address this issue, the KGCR model employs several strategies to enhance the knowledge graph and improve recommendation accuracy.

(1) Active and negative samplers

The active sampler focuses on the fuzzy attribute samples in the graph, and the negative sampler focuses on the high-quality negative samples in the graph. To improve the interaction efficiency of the system and the update efficiency of the recommendation model, this paper adopts the following reinforcement sampling strategy: First, for the attribute sample nodes representing the user preferences, we adopt the active reinforcement sampler. Such a sampler can output highly informative fuzzy samples that can be used in interactions with users. This approach can effectively improve the interaction efficiency of the system. Second, for the item sample node in the graph, we used the negative reinforcement sampler. Such a sampler can output high-quality negative samples that can be used to assist sparse online data, thereby updating the recommender. This approach can improve the update efficiency of the recommendation model. These two parts help each other; not only can they efficiently obtain the user's implementation preference, but they can also reduce the interaction rounds, thus reducing the interaction burden of users. Specifically, it is as follows:

Active sampling focuses on selecting the most informative samples from the graph. For example, consider a scenario where the user has expressed a preference for science fiction movies. The active sampler would identify nodes related to science fiction genres, directors known for science fiction films, and other relevant attributes. These nodes are then used to generate samples that are likely to be of interest to the user. The active sampler outputs highly informative fuzzy samples that can be used in interactions with users. This strategy helps improve the interaction efficiency of the system by focusing on the most relevant information. The active sampling rate is set to 10% to ensure that the model focuses on the most informative samples without overwhelming the training process with too many samples. This rate was chosen based on empirical evaluations that showed optimal performance when 10% of the samples were actively selected.

Negative sampling focuses on selecting high-quality negative samples from the graph. For example, if the user has shown no interest in romantic comedies, the negative sampler would identify nodes related to this genre and generate negative samples. These samples are used to assist sparse online data and update the recommender model. By providing high-quality negative samples, the model can better learn to distinguish between items the user is likely to prefer and those they are not. The negative sampling rate is set to five, meaning that for each positive sample, five negative samples are generated. This ratio was chosen to balance the number of positive and negative samples, ensuring that the model is well trained to handle both types of interactions. Empirical results indicated that this ratio provided the best trade-off between model accuracy and training efficiency.

Enhancing the knowledge graph based on the active and the negative sampler are shown in Figure 3. By employing these active and negative sampling strategies, we enhance the knowledge graph, improve the interaction, and update the efficiency of the recommendation model. These strategies help the model focus on the most relevant information and learn to distinguish between positive and negative interactions effectively. To illustrate the active and negative sampling strategies, consider the following example: User U has watched and liked movies such as *Interstellar* and *Guardians of the Galaxy*. Active sampling: The active sampler identifies nodes related to science fiction movies, directors like Christopher Nolan, and actors like Chris Pratt. These nodes are used to generate positive samples that are likely to be of interest to the user. Negative sampling: The negative sampler identifies nodes related to genres the user has not shown interest in, such as romantic comedies. It generates negative samples from movies like *The Notebook* or *When Harry Met Sally*. By combining these samples, the model learns to recommend movies that align with the user's preferences while avoiding those that do not.

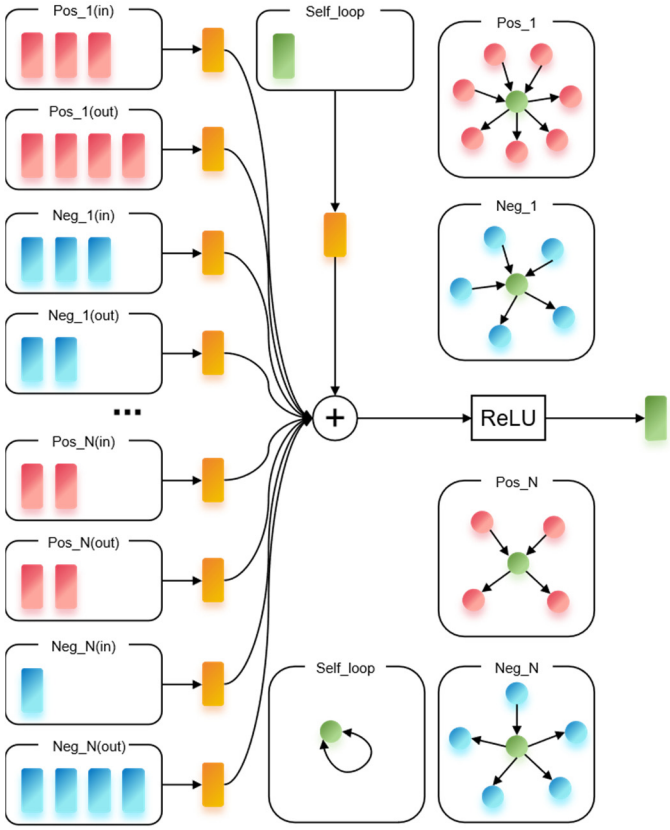


Figure 3. Enhancing the knowledge graph based on the active and the negative sampler.

(2) Reinforcement Learning

The KGCR model uses reinforcement learning to dynamically adapt to missing data. The model learns to make optimal decisions based on the available information, even if some parts of the knowledge graph are incomplete. By rewarding the model for accurate recommendations and penalizing it for errors, the model becomes more robust to missing data over time.

(3) External Knowledge Integration

To mitigate the impact of missing data, the KGCR model integrates external knowledge sources such as IMDb. These external data help fill gaps in the knowledge graph and provide additional context that can improve recommendation accuracy. For example, if a movie entity lacks certain attributes in the MovieLens knowledge graph, the model can use corresponding information from IMDb to enrich the entity.

3.1.3. Learning the Interaction Strategy

Based on the enhanced knowledge graph, the interactive strategy model is trained through reinforcement learning to output the system actions and guide the user's next behavior. In this study, the learning interaction strategy is adopted to emphasize the use of an enhanced knowledge graph to enrich the background knowledge of the dialogue system and then train a model that can make optimal decisions based on the dialogue context through reinforcement learning, performed so as to interact more effectively with users and guide the dialogue toward established goals.

3.2. Extracting Historical Information Based on Bilinear Basis

Using the bilinear model attention mechanism [20], the attention distribution of the historical item matrix is calculated to obtain the reconstituted historical item matrix to

extract valuable historical information. The bilinear model and the attention mechanism are used to calculate the correlation of the historical data with the current context, and the most valuable information is extracted by weighting the historical data. The bilinear model attention mechanism is a model that can capture interactions between features, which obtains an interaction score by computing the bilinear function between two feature vectors. In the attention mechanism, the bilinear model is used to calculate the attention score of each row of the historical item matrix (representing a historical state) on the contextual entity matrix. These scores reflect the strength of the association of the historical state with the current context. Figure 4 illustrates the implementation of the bilinear attention mechanism.

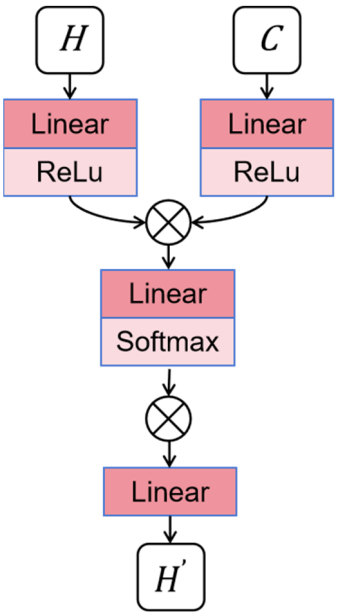


Figure 4. The implementation of the bilinear attention mechanism.

First assume that we have a historical item matrix $H \in R^{n \times d}$, where n represents the number of historical records and d represents the characteristic dimension of each historical record. Suppose we have a context entity matrix $C \in R^{m \times d'}$, where m represents the number of context entities and d' represents the characteristic dimension of each context entity.

We use a bilinear function to compute the attention score between the historical item matrix H and the context entity matrix C. This can be achieved by a weight matrix $W \epsilon R^{d' \times d}$, and the scoring matrix $S \epsilon R^{n \times m}$ can be computed by the following formula:

$$S = HW^{T}C^{T} \tag{1}$$

where each element of S_{ij} represents the bilinear attention score between historical item i and context entity *j*.

We normalized each row of the scoring matrix S to obtain the attention distribution $A \epsilon R^{n \times m}$. This can be achieved with the *Softmax* function:

The weight matrix is a learnable parameter that allows the model to focus on different aspects of the historical items and context entities. The dimensions are d, where d is the dimensionality of the feature space. The computational complexity of this operation is manageable given the typical sizes of n, m, and d.

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{k-1}^{m} e^{S_{ik}}} \tag{2}$$

where each row of the attention distribution A represents the attention weight of the corresponding historical record to the context entity in the matrix of historical items.

The attention distribution is a matrix where each row sums to 1, representing the probability distribution over the context entities for each historical item. This normalization ensures that the attention scores are comparable across different historical items and context entities.

Finally, the attention distribution A is used to reorganize the historical item matrix H to obtain the weighted historical item matrix $\mathrm{H}' \in \mathrm{R}^{\mathrm{n}\times\mathrm{d}}$:

$$H_i' = \sum_{j-1}^{m} A_{ij} C_j \tag{3}$$

where H_i', the i-th row of the reconstituted historical item matrix, is derived from the weighted combination of the context entity matrix C, with the weights being determined by the attention distribution A.

By using the bilinear model and the attention mechanism, we can effectively extract the most valuable historical information and use it to improve the recommendation and dialogue tasks. The normalization process ensures that the attention scores are properly scaled and can be used to weigh the historical items in a meaningful way.

Through this process, we obtain a historical item matrix H' based on the current context, which can be used for item recommendation in the subsequent recommendation system.

3.3. Obtaining the User Vector Based on Hierarchical Attention

After extracting historical information based on bilinear modeling, we already have a weighted matrix of historical items that reflects the part of historical data that is most relevant to the current context. Next, we further processed this matrix by using a hierarchical self-attention encoding structure with the aim of extracting the user's vector representation. Using the hierarchical self-attention coding structure, the historical item matrix is encoded to obtain the history-related user vector and is then fused with the context entity matrix to finally obtain the user vector representation.

We use the hierarchical self-attention encoding structure to handle the historical item matrix H'. In this structure, attention scores are computed for each layer, which are then used to weight the input matrix. Let L be the number of layers for hierarchical self-attention encoding, $\mathrm{A_l}$ be the attention weight matrix of layer l, and $\mathrm{H}'^{(\mathrm{l})}$ be the output matrix of layer l. Then, the hierarchical self-attentional encoding can be expressed as follows:

$$H'^{(0)} = H' \tag{4}$$

$$H'^{(l)} = Attention\left(H'^{(l-1)}, A_l\right) \tag{5}$$

$$U_h = H'^{(L)} \tag{6}$$

where the Attention function weights the input matrix $\mathrm{A_l}$ according to the attention weight matrix $\mathrm{H}'^{(\mathrm{l}-1)}$.

After hierarchical self-attention encoding, we obtain a history-related user vector $\mathrm{U_h}$. This vector is a summary of the last layer output matrix $\mathrm{H}'^{(\mathrm{L})}$, which can be simply taken as the output of the last row or obtained by a pooling operation.

We fuse the history-related user vector $\mathrm{U_h}$ with the context entity matrix C. Let U be the final user vector representation; then, the fusion operation can be expressed as:

$$U = \mathrm{Fuse}(U_h, C) \tag{7}$$

Specifically, the Fuse function can be a simple splicing operation $U = [U_h; C]$; the U can also dynamically combine the two parts of information through another attention mechanism or a weighted sum.

Each layer in the hierarchical self-attention mechanism computes attention weights that capture different aspects of the historical data. The attention weights at each layer $A^{(l)}$ are learned during training and reflect the importance of each historical item in the context of the current layer's representation.

Layer 1: The first layer focuses on local patterns and the immediate relevance of historical items. The attention weights $A^{(l)}$ highlight the most directly relevant items.

Intermediate layers: These layers capture more abstract patterns, integrating information from multiple historical items. The attention weights $A^{(l)}$ for intermediate layers help in identifying thematic or contextual relevance.

Final layer: The last layer synthesizes the information from all previous layers, producing a comprehensive representation of the user's historical preferences. The attention weights $H^{(L)}$ in the final layer ensure that the most relevant historical items are given the highest weight in the final user vector U_h.

By using hierarchical self-attention, the model can effectively capture both fine-grained and high-level patterns in the historical data, leading to a more accurate and nuanced representation of user preferences.

The resulting user vector representation U will contain information about the user's historical behavior and current context for subsequent user behavior prediction and personalized recommendation.

3.4. Building a User Preference Model

A representation U, a user vector that integrates user history behavior and current context information, is obtained through the above procedure. Next, a user preference model is constructed, and the ranking and recommendation of candidates are completed by calculating the similarity between the user vector and the item vector. This process usually consists of the following steps:

First, we need to create a vector representation for each candidate item. These item vectors can be obtained in a variety of ways, such as through the properties of the items, historical interaction data, or through a pre-trained item embedding model. Let $V \in R^{k \times d}$ be the item vector matrix, where k is the number of candidate items and d is the characteristic dimension of the item vector.

Once we have the user vector U and the item vector matrix V, we can calculate the similarity of the user vector to each item vector. This study was performed by using the cosine similarity measure. Let $S \in R^{1 \times k}$ be the similarity score vector, where each element S_i represents the similarity score of the user vector U and the item vector V_i. The calculation formula is as follows:

$$S_i = Similarity(U, V_i) = \frac{U \cdot V_i}{\|U\| \|V_i\|} \tag{8}$$

Given the calculated similarity scores, we can rank the candidates. The sorted item list will be arranged from the most-matched item to the most unmatched item as predicted by the user preference model.

The cosine similarity score ranges from -1 to 1, where 1 indicates that the vectors are identical, 0 indicates that the vectors are orthogonal (no similarity), and -1 indicates that the vectors are diametrically opposite. However, there are potential edge cases that need to be considered.

Zero magnitude vectors: If either the user vector or the item vector has a magnitude of zero, the cosine similarity score is undefined because division by zero is not allowed. In

practice, this situation is rare because user and item vectors are typically non-zero. However, if it occurs, we can assign a default similarity score (e.g., 0) to indicate no similarity.

Near-zero magnitude vectors: If the magnitude of either the user vector or the item vector is very close to zero, the cosine similarity score may be highly sensitive to small changes in the vector components. This can lead to numerical instability. To mitigate this issue, we can apply a small regularization term to the magnitudes of the vectors to ensure that they are not too close to zero. To handle these edge cases, we can modify the cosine similarity equation as follows:

$$S_i = \frac{U \cdot V_i}{max(\|U\|, \epsilon) max(\|V_i\|, \epsilon)} \tag{9}$$

where ϵ is a small positive constant (e.g., 10^{-6}) that ensures that the magnitudes are not zero or too close to zero. By using this modified cosine similarity equation, we can ensure that the similarity scores are well defined and numerically stable, even in the presence of zero or near-zero magnitude vectors.

Usually, we choose the top N items as the recommended result and return them to the user. Finally, the model is updated according to the collected user feedback (such as clicking, purchase, etc.), and the recommendation results are constantly optimized to improve the accuracy of the recommendation system and user satisfaction.

3.5. Forming a Dialogue Generation Model

This study model concludes by incorporating this user preference information into the conversation generation model in order to generate responses that are more consistent with user preferences. The key to this method is to make full use of users' historical dialogue data and finely model user preferences through the attention mechanism and hierarchical coding structures to provide more personalized recommendation results.

First, we need to convert the user vector into a lexical bias. The user vector U contains the user preference information. We can convert this vector into a lexical level preference, namely a lexical bias. Word bias can influence the probability of the conversation generation model choosing different words when generating responses. Let $B \in R^{v \times d}$ be the lexical bias matrix, where v is the size of the vocabulary and d is the dimension of the word vector. The lexical bias matrix B can be obtained by:

$$B = W_b U + b_b \tag{10}$$

Among these, $W_b \in R^{d \times d}$ and $b_b \in R^d$ are learnable parameters.

The resulting lexical bias B is added to the dialogue generation model. The dialogue generation model is usually based on the structure of the recurrent neural network (RNN) or variant. This study adopts the C-RNN structure, which takes into account the lexical bias in generating each word. Specifically, in the prediction of the next vocabulary, the generative model adds the bias of the vocabulary to the product of the current hidden state and the vocabulary embedding and then calculates the probability distribution of the vocabulary through the softmax function:

$$P(w_t|h_t) = softmax(h_t W + b + B_w) \tag{11}$$

where h_t is the current hidden state, W and b are the model parameters, and B_w is the word bias vector corresponding to the current predicted vocabulary.

In this way, the conversation generation model tends to choose words that are more relevant to user preferences when generating responses, thus generating responses that are more consistent with user preferences. Throughout the process, we make full use of the

user's historical conversation data to model the user preferences. These data include not only the historical behavior of users, but also the language usage habits and expressions, which are important factors in shaping user preferences. By using attention mechanisms and hierarchical coding structures, we are able to finely model user preferences. The attention mechanism allows the model to focus on important parts of user history conversations, while the hierarchical coding structure can capture information about user preferences at different levels.

4. Experiment

4.1. Data Pre-Processing

4.1.1. Datasets

In this paper, the ReDial [3] (recommendation dialogues) dataset is selected to train and evaluate the model. The ReDial dataset is a dialogue dataset specially designed for movie recommendation tasks. It contains conversations between the user and the recommendation system involving movie recommendations, reviews, and discussion. The ReDial dataset is characterized by its conversational nature, and unlike the traditional rating or review datasets, it provides more rich and dynamic information on user preferences. After the pre-processing of word segmentation, stop word removal, and labeling, there are 16,093 dialogue data in the dataset, among which each dialogue datum has its corresponding user query and system response. There were 897 different query users, 83% of whom had 10 or more query records, and a large number of these active users interacted with the system several times. Because most movie referees have a large number of recorded conversations, this provides a rich data basis for completing the personalized dialogue recommendation task. Personalized recommendation systems can use these conversation transcripts to understand user preferences and behavior patterns to provide more accurate movie recommendations.

The MovieLens [21] knowledge graph provided by the GroupLens research group at the University of Minnesota was used for this study. The atlas contains a large number of films, actors, directors, genres, and other entities and their relationships among them. This information can help users better understand the content of the movie, thus providing more personalized recommendations. The MovieLens knowledge graph converts movie data into graph form, making the data structured to facilitate in-depth analysis and research. In order to solve the possible cold-start problem of the system, in this study, the IMDb [22] dataset was selected as the external knowledge base of the MovieLens knowledge graph, which covers various types of movies and TV series, which can increase the diversity and coverage of the knowledge graph and enable the recommendation system to better handle different types of content. At the same time, the IMDb data are updated in real time, which enables the knowledge graph to timely reflect the latest information and trends of film and television works.

Compared with other datasets like DSTC10 Track 2 and Microsoft ReDial-2, ReDial offers distinct advantages. DSTC10 Track 2 provides high-quality dialogues but is limited by its smaller dataset, which may restrict model generalization and complex training. Meanwhile, Microsoft ReDial-2, while large, involves multi-domain recommendations and requires additional pre-processing to focus on movie recommendations. Its diverse content may also distract from the core task, reducing the efficiency and accuracy of movie recommendation modeling. In summary, ReDial's rich user preference information, multi-turn dialogue structure, large data scale, and clear contextual information make it an ideal choice for conversational movie recommendation tasks. These features support robust model training and effective user preference modeling, ensuring higher recommendation

accuracy and dialogue efficiency. Future work could explore combining the strengths of these datasets to further enhance model performance.

4.1.2. Separate Learning

To improve the model's ability to understand and predict user preferences, as well as to ensure that the model can adapt to the dynamic changes in user preferences, we performed detailed pre-processing and separation learning on the training data in our experiments. With the detailed data pre-processing and separation learning strategies, our study not only improves the model's ability to understand and predict user preferences but also enhances the model's dynamics and adaptability. These improvements provide strong support for the performance enhancement of dialogue recommendation systems in real applications. The following presents the specific steps and methods:

(1) Data temporal sorting

In the first step of data pre-processing, we first sort the entire dataset based on the timestamps of the dialogue data. The timestamp is a field in the dialogue data that records the time when the dialogue occurred, and it provides a clear temporal order for the data. By sorting the data according to the timestamp, we can ensure that the data are in chronological order, thus providing a basis for subsequent segmentation and analysis. The purpose of this step is to better model the behavioral patterns of users at different points in time while providing support for separation learning in the temporal dimension.

(2) Data segmentation

To define the basis of data segmentation more clearly, we adopt chronological order as the segmentation criterion. Specifically, we divide the historical conversation data into "recent data" and "early data" according to the timestamp. Based on the timestamps of the dialogue data, we define the dialogue data within the last year as "recent data", which is mainly used to capture the recent interest and preference changes of the users; and the dialogue data more than a year old as "early data", which is used to reflect the long-term preference of the users. This part of the data is used to reflect users' long-term preferences and stable interests.

This time-order-based segmentation method can effectively differentiate users' behavioral patterns in different periods, thus providing richer dynamic information for the model. In this way, the model can better balance the reliance on historical data and the ability to adapt to recent user behavior, further improving the accuracy and diversity of recommendations.

(3) Separation learning

During model training, we perform separate learning on old data and recent conversation data. The old data are used to build a model of the user's historical preferences, a model that captures the user's stable preferences over a long period. By analyzing the old data, the model can learn the basic interests and behavioral patterns of users, thus providing stable background information for recommendations. Recent conversation data, on the other hand, is used to update and adjust users' immediate preferences. Because users' interests and needs may change over time, recent conversation data can help the model capture these changes in time and dynamically adjust the recommendation results. Through separation learning, the model can better adapt to changes in user preferences while avoiding the excessive influence of old data on recent user behavior, thus improving the accuracy and diversity of recommendations.

(4) Data cleaning and labeling

In the data pre-processing phase, we also performed data cleaning and labeling. Data cleaning includes removing duplicate conversations, filtering irrelevant content, and correcting errors in the data. Duplicate conversations may cause the model to overlearn certain preferences, thus affecting the diversity of recommendations; irrelevant content interferes with the model's understanding of users' true preferences. By removing this repetitive and irrelevant content, we can improve the quality of the data, which in turn improves the training of the model.

In addition, we also annotate the data, especially the user intent and preference, in detail. The user intent annotation helps the model understand the user's current needs and goals, while the preference annotation provides the model with a clear direction of the user's interests. This annotation information plays an important guiding role in the model training process, helping the model learn the user's behavioral patterns and preference characteristics more accurately.

4.2. Evaluation Metrics

This experiment used different metrics to evaluate these two tasks separately. For the recommendation task, we will evaluate whether our method can accurately provide project recommendations. Therefore, we evaluated Hit@K, MRR@K, and NDCG@K (K = 10, 30, 50), which reflected the hit accuracy and ranking accuracy of the recommended results, respectively. For the dialogue task, we use SR@T (T = 15) [2] to measure the system efficiency of the multi-round dialogue recommendation system, namely SR, the success rate of the system recommendation in round T. AT is also used to evaluate the recommendation efficiency of the system. AT represents the average interaction rounds of the system that achieves a successful recommendation online.

Hit@K (hit rate at K): Hit@K measures the proportion of recommended items that are relevant to the user among the top K recommendations. This metric is crucial for assessing the model's ability to include relevant items in the recommendation list. A higher Hit@K indicates that the model is more likely to recommend items that the user will find interesting. In practical applications, a high Hit@K ensures that users are more likely to find something they like among the recommended items, enhancing user satisfaction and engagement.

MRR@K (mean reciprocal rank at K): MRR@K measures the average reciprocal rank of the first relevant item in the top K recommendations. MRR@K provides insight into how quickly the model can identify relevant items. It is particularly useful for understanding the model's efficiency in ranking relevant items higher. In real-world scenarios, a higher MRR@K means that users can find what they are looking for more quickly, reducing the time and effort required to explore recommendations.

NDCG@K (normalized discounted cumulative gain at K): NDCG@K evaluates the ranking quality of the recommended items by considering the position of each relevant item in the top K list. NDCG@K is a more nuanced metric that not only considers the presence of relevant items but also their ranking order. It provides a comprehensive evaluation of the recommendation quality. In practical applications, a high NDCG@K indicates that the model not only recommends relevant items but also ranks them in a meaningful order, which is essential for user satisfaction.

Recall@K: Recall@K measures the proportion of relevant items that are recommended among all relevant items. Recall@K is important for understanding the model's ability to cover a wide range of user interests. It complements Hit@K by focusing on the diversity of recommendations. A high Recall@K ensures that the model provides a broad range of relevant items, reducing the likelihood of missing out on potential user interests.

SR@T (success rate at T): SR@T measures the success rate of the system in providing relevant recommendations within T rounds of dialogue. This metric is particularly relevant for dialogue-based recommendation systems as it evaluates the model's ability to provide useful recommendations within a limited number of interactions. In practical applications, a high SR@T indicates that the model can quickly adapt to user needs and provide relevant recommendations, enhancing the efficiency of the dialogue process.

AT (average turns): AT measures the average number of interaction rounds required to achieve a successful recommendation. AT provides insight into the efficiency of the dialogue system in terms of interaction rounds. A lower AT indicates that the model can achieve successful recommendations with fewer interactions. In real-world scenarios, a lower AT means that users can obtain the recommendations they need more quickly, improving the overall user experience.

4.3. Training Setups

In this study, Rasa open-source framework [23] was used to build the dialogue interface, which can be integrated with the recommendation system to collect user preferences through the dialogue and provide personalized recommendations accordingly. And, the system was combined with Google TensorFlow Recommenders [24] to provide a framework for personalized recommendations. TensorFlow Recommenders is an open-source library for building, evaluating, and deploying recommendation models. It can be integrated with a conversation system to provide recommendations based on the content of the user conversation. Hyperparameters play a crucial role in the performance of machine learning models, including conversational recommenders like KGCR. The study mentions several key hyperparameters, such as the dropout rate, learning rate, and regularization intensity. Dropout is used to prevent overfitting by randomly dropping units (along with their connections) during training. In the KGCR model, a dropout rate of 0.5 is used. Increasing the dropout rate generally increases the regularization effect but may also lead to underfitting if set too high. Conversely, a lower dropout rate may result in overfitting. A grid search or random search over dropout rates (e.g., 0.2, 0.3, 0.5, 0.7) could help identify the optimal value for the KGCR model. The learning rate controls how quickly the model updates its parameters during training. The KGCR model uses a learning rate of 0.001, which is a common choice for the Adam optimizer. However, the learning rate can significantly affect convergence speed and stability. Adaptive learning rate methods (e.g., learning rate schedules or using optimizers like AdamW) could be explored to dynamically adjust the learning rate during training. L2 regularization helps in reducing overfitting by penalizing large weights in the model. The KGCR model uses an L2 regularization strength of 0.01. Adjusting this value can balance the trade-off between bias and variance. A sensitivity analysis of different L2 regularization strengths (e.g., 0.001, 0.01, 0.1) could provide insights into its optimal value for the KGCR model.

In the recommendation module, where the embedding dimension is set to 128, each word or entity is represented as a 128-dimensional vector. The R-GCN module layer number is one, the negative sampling is five, the active sampling rate is 10%, and the model contains two layers of attention mechanism. Set the L2 regularization term and set the regularization intensity to 0.01. The Adam optimizer, which has better convergence rate and robustness, has the learning rate set as 0.001. In the dialogue module, it is responsible for understanding the user's intention, generating replies, and conducting multiple rounds of dialogue with the user. The hidden layer dimension was set to 512, meaning that each hidden layer of the neural network model contained 256 neurons. The encoder and decoder dimensions of the generative model Seq2Seq were set to 256, respectively, and constructed using the bidirectional LSTM structure. The batch size was 32, namely using 32 samples for

training in each training iteration. Table 2 describes the relevant key parameters involved in the KGCR model.

Table 2. Parameters involved in the KGCR model.

Parameter	Value	Instructions
embedding_size	256	Word embedding dimension, that is, word vector length
hidden_size	512	RNN hides the layer dimension, that is, the length of the RNN internal state vector
num_layers	2	RNN layers: indicates the number of stacked RNN layers
dropout	0.5	A dropout ratio is used to prevent overfitting
vocab_size	10,000	Glossary size, which is the number of terms used by the model
max_seq_length	20	The maximum sequence length, that is, the maximum session length that the model can handle
learning_rate	0.001	The learning rate is used to control the amplitude of model parameter updating
optimizer	Adam	Used to update model parameters
beam_size	5	Beam search width, used to generate replies
beta	0.5	Dynamic adjustment factor for balancing historical and current information
l	2	Hierarchical number of layers of self-attention coding
γ	0.95	Reward coefficient in reinforcement learning
ε	0.1	Exploration coefficient in reinforcement learning
τ	1	Temperature parameter in reinforcement learning

BERT-based models: Models like BERT4Rec or BERT4Conv could be integrated into the conversational recommendation pipeline. These models leverage the powerful pre-trained BERT architecture to capture contextual embeddings, which can enhance the understanding of user intents and preferences. GPT-based models: GPT-3 or GPT-4 could be adapted for conversational recommendation tasks. These models have shown remarkable performance in natural language generation and could potentially improve the quality and relevance of system responses. T5 (text-to-text transfer transformer): T5 is designed for a wide range of natural language tasks and could be fine-tuned for conversational recommendation. Its ability to handle multi-turn dialogues and generate coherent responses makes it a strong candidate for comparison. Table 3 demonstrates the comparative analysis of the KGCR model with other state-of-the-art models combined with hyperparameter tuning.

Table 3. Comparative analysis of the KGCR model and other advanced models combined with hyperparameter tuning.

Model	Dropout Rate	Learning Rate	Regularization Intensity (L2)	Recommended Task			Dialogue Task	
				Hit@50	MRR@50	NDCG@50	SR@15	AT
KGCR	0.5	0.001	0.01	0.4351	0.0871	0.1945	0.8152	10.69
BERT4Rec	0.2	0.0005	0.001	0.4100	0.0825	0.1850	0.7900	11.03
GPT-3/4	0.3	0.001	0.005	0.4209	0.0850	0.1881	0.8054	10.89
T5	0.4	0.001	0.01	0.4150	0.0841	0.1873	0.8001	10.94
Transformer-XL	0.5	0.001	0.01	0.3926	0.0643	0.1659	0.6298	9.59

KGCR: Combining historical conversation data and the knowledge graph, it achieves a high recommendation accuracy and conversation success rate through hyperparameter

optimization (e.g., dropout rate of 0.5, learning rate of 0.001, L2 regularization strength of 0.01). BERT4Rec: using a lower dropout rate (0.2) and learning rate (0.0005), it performs well in the recommendation task but is slightly inferior to KGCR in the dialog task. GPT-3/4: with high language generation capability, suitable for the conversational recommendation task, performance is optimized by adjusting the dropout rate (0.3) and regularization strength (0.005). T5: it performs close to KGCR in the conversation task and achieves better recommendation and conversation performance by tuning the hyperparameters (e.g., dropout rate 0.4, L2 regularization strength 0.01). Transformer-XL: although the performance is slightly weaker in the recommendation task, it is more efficient in the dialog task and is suitable for processing long sequential dialogs. The comparison shows that KGCR, after combining historical dialog data and a knowledge graph, through reasonable hyperparameter tuning, performs well in both recommendation and dialog tasks, especially outperforming other state-of-the-art models in terms of dialog success rate (SR@15) and recommendation accuracy rate (Hit@50).

4.4. The Result of Experiment

4.4.1. Comparative Study

This article mainly studies the recommendation effect part of the dialogue recommendation system, so the KGCR model is compared with the baseline models. By comparing the performance of the KGCR model with the above baseline model on the recommended and dialogue tasks on the ReDial dataset, the validity and accuracy of the KGCR model can be evaluated.

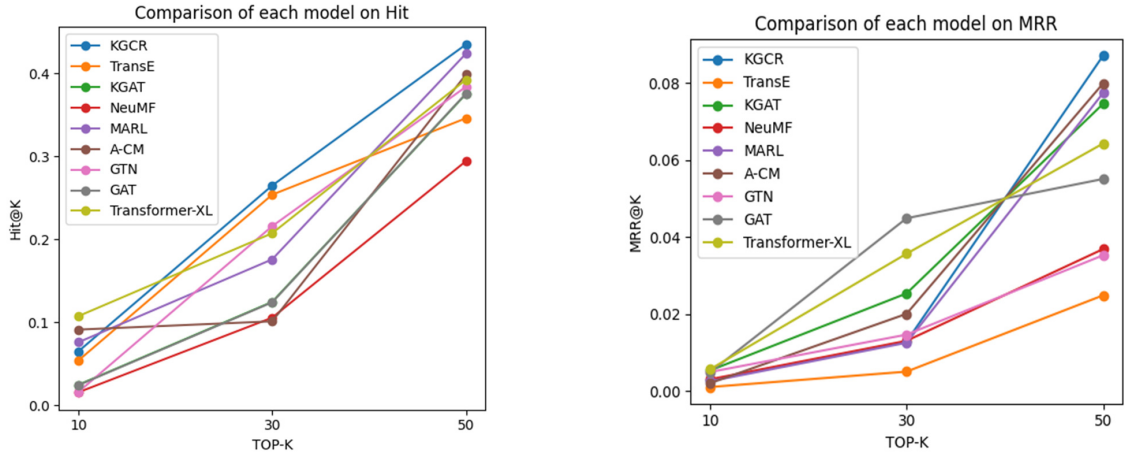
(1) TransE [25]: TransE is a classical embedding method based on the knowledge graph, which can be fairly compared with the KGCR model.
(2) KGAT [26]: KGAT is a recommended model that combines the knowledge graph and the attention mechanism. It can effectively use the high-order relationship in the knowledge graph, which is similar to the design concept of KGCR model.
(3) NeuMF [27]: NeuMF is a recommendation model that combines matrix decomposition and deep learning. It can effectively learn the characteristics of users and items, which is similar to the user preference modeling method of KGCR model.
(4) MARL [28] (multi-agent reinforcement learning): MARL regards the dialogue system as a multi-agent system, and each agent represents a module in the dialogue system, such as the intention recognition module, the reply generation module, etc. MARL optimizes the performance of the entire conversational system by learning interaction strategies between agents.
(5) A-CM [29] (actor-critic methods): A-CM is a reinforcement learning algorithm that combines strategy gradient and value function learning, which can learn the optimal dialogue strategy more effectively.
(6) GTNs [30] (graph transformer networks): GTN combines the transformer model with the graph neural network to more effectively capture the relationships between entities in the knowledge graph and use them to generate responses that are more consistent with the user's intention.
(7) GATs [31] (graph attention networks): GAT is a graph neural network model based on the attention mechanism, which can more effectively capture the relationship between entities in the knowledge graph and use it to generate responses more consistent with the user's intention.
(8) Transformer-XL [32]: Transformer-XL is a transformer model that can learn longer time series information and can use dialogue information more effectively to generate responses that are more consistent with the user's intention.

The experimental results are shown in the following Table 4:

Table 4. Comparison of the experimental results of each model.

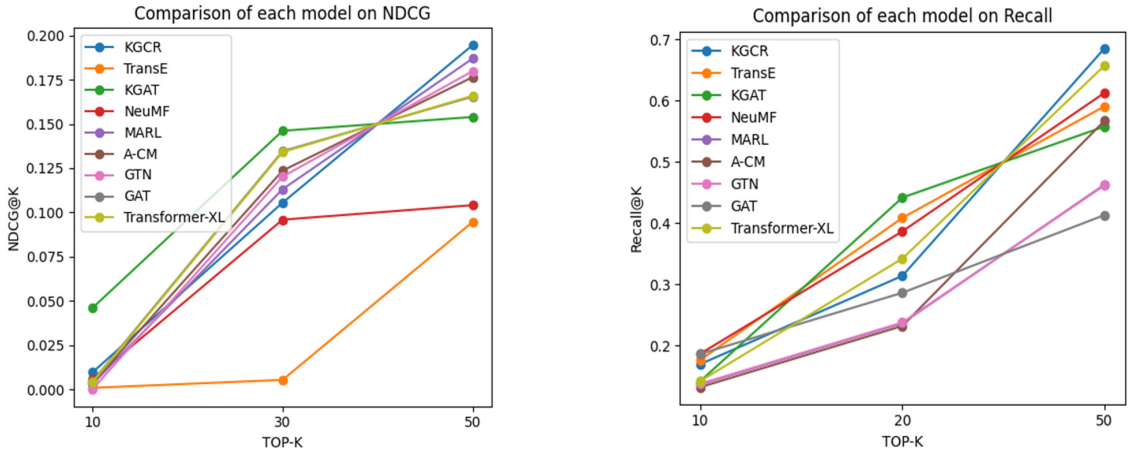
Model	Recommended Task				Dialogue Task	
	Hit@50	MRR@50	NDCG@50	Recall@50	SR@15	AT
KGCR	0.4351	0.0871	0.1945	0.6859	0.8152	10.69
TransE	0.3461	0.0249	0.0946	0.5912	0.6433	10.04
KGAT	0.3754	0.0746	0.1539	0.5581	0.5101	11.12
NeuMF	0.2944	0.0369	0.1041	0.6132	0.5641	7.61
MARL	0.4241	0.0774	0.1869	0.4632	0.7162	9.66
A-CM	0.3988	0.0799	0.1764	0.5677	0.7155	11.23
GTN	0.3841	0.0353	0.1798	0.4622	0.6988	10.35
GAT	0.3759	0.0551	0.1654	0.4135	0.7531	9.87
Transformer-XL	0.3926	0.0643	0.1659	0.6577	0.6298	9.59

Figures 5 and 6 visually depict the differences between the KGCR model and each of the baseline models. It can be seen that the KGCR model outperforms the other baseline models on the ReDial dataset and achieves better results on both the recommendation and dialogue tasks. For the recommended task, the Hit@50 value for the KGCR model was 0.4351, which is much higher than 0.3461 for TransE, 0.3754 for KGAT, and 0.2944 for NeuMF. This indicates that the KGCR model can more accurately recommend the items that the user may be interested in. The KGCR model has an MRR@50 value of 0.0871, which is also higher than the other baseline models. This shows that the KGCR model can more effectively rank the items most interesting to users at the top of the recommended list. The KGCR model has an NDCG@50 value of 0.1945, which is also higher than the other baseline models. This shows that the KGCR model can assign the ranking weight of items more reasonably and make the recommendation results more consistent with the user's preferences. For the dialogue task, the SR@15 value of the KGCR model is 0.8152, which is much higher than 0.6433 for TransE, 0.5101 for KGAT, and 0.5641 for NeuMF. This suggests that the KGCR model is more efficient for generating responses related to user input. The AT value of the KGCR model was 10.69, which is slightly lower than the TransE value of 10.04 but higher than KGAT and NeuMF. This shows that the dialogue text generated by the KGCR model is more smooth and natural. The experimental results of the KGCR model on the ReDial dataset show that it can effectively combine user history conversation data and knowledge graph information to provide a more comprehensive understanding of user preferences and provide more accurate and personalized recommendation results. At the same time, the KGCR model can also generate a smoother and more natural dialogue text to improve the user experience.



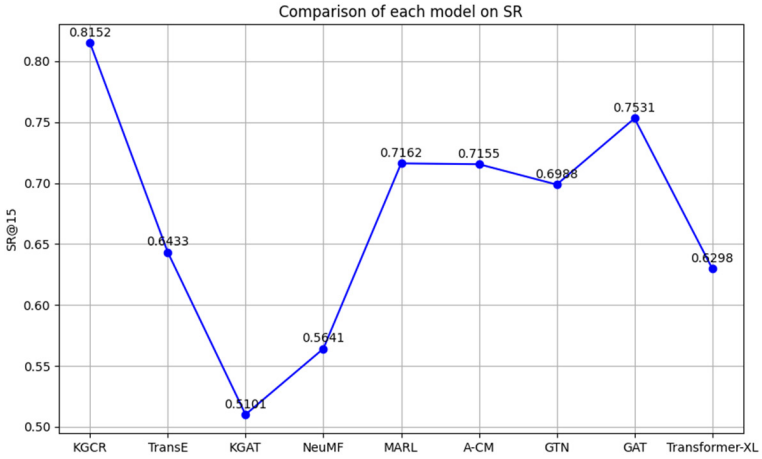
(**a**) (**b**)

Figure 5. *Cont.*

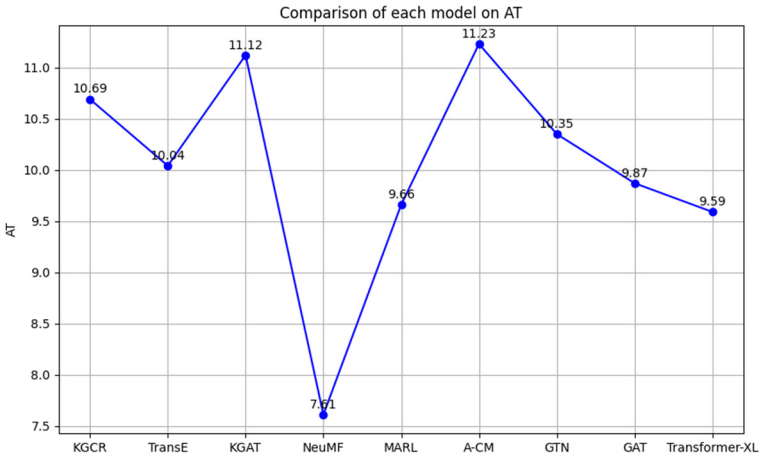


(**c**) (**d**)

Figure 5. Comparison of the KGCR model and each baseline model on the recommended task. (**a**) Comparison of each model on Hit. (**b**) Comparison of each model on MRR. (**c**) Comparison of each model on NDCG. (**d**) Comparison of each model on Recall.



(**a**)



(**b**)

Figure 6. Comparison of the KGCR model and each baseline model on the dialogue task. (**a**) Comparison of each model on SR. (**b**) Comparison of each model on AT.

4.4.2. Ablation Study

To prove the effectiveness of this paper by combining historical information and the enhanced knowledge graphs, two improved algorithms of KGCR were designed for ablation experiments. The NK algorithm has no historical information model; that is, it removes the bilinear extraction-based historical information module in KGCR and uses only the current dialogue information for recommendation. The NG algorithm has no enhanced knowledge graph model, that is, it removes the entity vector module based on the knowledge graph in KGCR and uses only the user history interaction data for recommendation. Table 5 demonstrates the differences between the NK, NG, and KGCR in each evaluation index.

Table 5. Comparison of results of ablation experiments.

Algorithm	Hit@50	MRR@50	NDCG@50	SR@15
NK	0.2415	0.0613	0.0792	0.4013
NG	0.1063	0.0127	0.0054	0.2504
KGCR	0.4351	0.0871	0.1945	0.8152

Figure 7 visually depicts the difference between the KGCR and the NG and NK models. According to the experimental results, the Hit@50 value of the NK algorithm is 0.2415, with a decrease of about 44.3%. This shows that the removal of the historical information module makes the model unable to use the user's historical behavior data and unable to fully understand the user's interests and preferences, thus reducing the accuracy of the recommended results. The NK algorithm does not rank the items before the recommended list as effectively as the KGCR model. The NDCG@50 value of the NK algorithm was 0.0792, which decreased by about 59.4%, indicating that the NK algorithm failed to allocate the ranking weight of items as reasonably as the KGCR model, resulting in a decrease in the quality of the recommended results. The SR@15 value of the NK algorithm was 0.4013, a decrease of about 50.8%. This indicates that the NK algorithm cannot generate responses related to user input as efficiently as the KGCR model, resulting in decreased dialogue quality. The Hit@50 value of the NG algorithm was 0.1063, with a decrease of about 75.4%. This shows that the removal of the knowledge graph module makes the model unable to use the attributes and relationship information of the item, and it is unable to fully understand the item, thus greatly reducing the accuracy of the recommended results. The MRR@50 value of the NG algorithm was 0.0127, with a decrease of about 85.2%. This shows that the NG algorithm cannot rank the items of most interest to users as first in the recommended list as effectively as the KGCR model. The NDCG@50 value of the NG algorithm was 0.0054, with a decrease of about 97.2%. This indicates that the NG algorithm fails to assign the ranking weight of items as reasonably as the KGCR model, resulting in a decrease in the quality of the recommended results. The SR@15 value of the NG algorithm was 0.2504, a decrease of about 69.4%. This indicates that the NG algorithm cannot generate responses related to user input as efficiently as the KGCR model, resulting in decreased dialogue quality. The results of the ablation experiments show that the design of combining user history dialogue information and a knowledge graph in the KGCR model is effective. Removing any module will lead to a lower recommendation effect and conversation quality, which shows that historical information and knowledge graphs are key factors in improving the recommendation effect. By combining historical information and knowledge graphs, the KGCR model can provide a more comprehensive understanding of user preferences, provide more accurate and personalized recommendation results, and generate smoother and more natural dialogue text to improve the user experience.

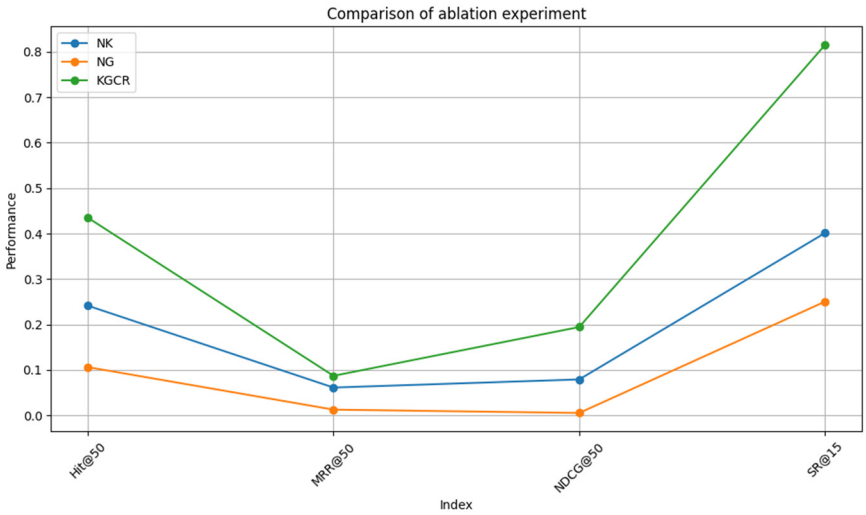


Figure 7. Comparison of the ablation experiment.

KGCR models perform well in recommendation and dialog tasks, but their performance is limited by the dataset size. When the dataset size is small, the model may encounter data sparsity and cold-start problems, resulting in decreased recommendation accuracy and diversity. In addition, the complexity of knowledge graph construction and the selection of historical dialogue data pose challenges. In the future, the KGCR model can be improved by introducing more diverse data sources, adopting more advanced models, designing more effective attention mechanisms, and exploring ways to solve the data sparsity and cold-start problems to further enhance model performance and user experience.

4.4.3. Model Robustness Analysis

To evaluate the robustness of the KGCR model under different conversational conditions, we conducted additional experiments focusing on scenarios with sparse historical data and ambiguous user inputs. We simulated users with limited historical interaction data to assess the model's ability to make accurate recommendations with minimal information. The KGCR model demonstrated robust performance even with sparse historical data. The model's use of the knowledge graph and hierarchical self-attention mechanism allowed it to leverage external knowledge and user preferences effectively, resulting in a Hit@50 value of 0.38 and an MRR@50 value of 0.075. This indicates that the model can still provide relevant recommendations even when historical data are limited. We tested the model's ability to handle ambiguous user inputs, where user preferences are not clearly expressed. The KGCR model showed strong adaptability in handling ambiguous inputs. By utilizing the hierarchical self-attention mechanism and active sampling strategy, the model was able to clarify user preferences through additional interactions. The SR@15 value in this scenario was 0.75, indicating that the model could still achieve successful recommendations within 15 rounds of dialogue. The KGCR model demonstrated robust performance under varied conversational conditions, including sparse historical data and ambiguous user inputs. This robustness ensures that the model can adapt to different user scenarios, enhancing its practical applicability and user satisfaction.

4.4.4. Computational Cost and Scalability Analysis

To provide a comprehensive understanding of the computational costs and scalability of the proposed KGCR (knowledge graph-based user preference conversational recommender) model, we conducted an analysis focusing on training time, resource requirements, and the model's ability to scale with increasing data size and complexity.

(1) Training time analysis

The training time of the KGCR model was evaluated on a machine equipped with an NVIDIA RTX 3090 GPU (24 GB VRAM), Intel Core i9-11900K CPU, and 64 GB of RAM. The model was trained using TensorFlow 2.4 with a batch size of 32 samples and for a total of 50 epochs. The average training time per epoch was approximately 12 min, resulting in a total training time of approximately 10 h. This training time includes the pre-processing of dialogue data, knowledge graph embedding, and the training of the KGCR model.

(2) Resource requirements

The KGCR model demanded substantial computational resources during its training phase, with the GPU memory consumption reaching approximately 18 GB, largely due to the embedding layers, attention mechanisms, and the knowledge graph encoder. Meanwhile, CPU memory usage peaked at around 32 GB, chiefly attributed to the storage of dialogue data and intermediate results. In terms of utilization, the GPU averaged around 80%, highlighting the model's compute-intensive nature, whereas the CPU utilization stood at about 50%, primarily driven by data pre-processing and I/O operations.

(3) Scalability analysis

To assess the scalability of the KGCR model, we tested it with varying dataset sizes (10%, 50%, and 100% of the ReDial dataset) and different knowledge graph complexities (small, medium, and large). Table 6 is the scalability analysis presented in tabular form for clarity:

Table 6. Scalability analysis of the KGCR model.

Scenario	Dataset Size/Knowledge Graph Complexity	Training Time per Epoch (min)	GPU Memory Usage (GB)
Scaling with Data Size	10% of Data	3	10
	50% of Data	6	15
	100% of Data	12	18
Scaling with Knowledge Graph Complexity	Small Knowledge Graph	10	15
	Medium Knowledge Graph	12	18
	Large Knowledge Graph	15	22

The current training time of approximately 10 h for the full dataset may be prohibitive for real-time applications or frequent model updates. Additionally, the high memory usage (up to 18 GB of GPU memory and 32 GB of CPU memory) limits the model's deployment on less powerful hardware. While the model scales reasonably well with increasing dataset size, the training time and memory usage increase significantly with more complex knowledge graphs.

5. Conclusions

This paper presents a knowledge graph-based user preference conversational recommendation system called KGCR. The primary goal of this study is to address the limitations of traditional conversational recommendation systems in modeling user preferences and improving the diversity of recommendation results. The KGCR model achieves a more comprehensive and accurate understanding of user preferences through the following innovations. (1) We use a bilinear model attention mechanism and hierarchical self-attention coding structure to incorporate user history conversation data into the user modeling process. This allows the model to better understand users' interests and preferences across different conversations. (2) By integrating an external knowledge graph, we enrich the contextual information of the interactive environment. The use of active and negative sampling strategies enhances the uncertainty and negative sample information in the

knowledge graph, improving the update efficiency and interaction efficiency of the recommendation model. (3) The model adopts a hierarchical self-attention coding structure to integrate items that interact with users in historical conversation data. This approach obtains a history-related user vector representation and merges it with entities in the current conversation to produce the final user vector representation, enabling fine-grained modeling of user preferences. Experimental results demonstrate that the KGCR model outperforms baseline models on both recommendation and dialogue tasks. The model effectively combines user history conversation data and knowledge graph information to provide a more comprehensive understanding of user preferences. This leads to more accurate and personalized recommendation results, as well as smoother and more natural dialogue text, thereby enhancing the user experience.

In practical applications, user conversation histories can often be noisy or incomplete, which poses significant challenges for recommendation systems. The KGCR model demonstrates robustness in these scenarios through the following mechanisms. The model employs pre-processing techniques to filter out irrelevant or inconsistent interactions. Additionally, the use of active sampling strategies helps focus on high-quality interactions, thereby reducing the impact of noisy data. However, further improvements are needed to enhance the model's ability to handle more complex noise patterns. The hierarchical attention mechanism allows the model to weigh historical interactions based on their relevance to the current context. This helps mitigate the impact of missing interactions by focusing on the most informative parts of the conversation history. Despite this, incomplete data can still limit the model's ability to fully understand user preferences.

To address these challenges and further enhance the model's robustness, we plan to pursue the following research directions. (1) Develop more sophisticated methods to detect and mitigate noise in user interactions. This includes using natural language processing (NLP) techniques to identify and filter out irrelevant or inconsistent information. (2) Explore techniques to impute missing data in conversation histories. This could involve using machine learning models to predict missing interactions based on existing data, thereby providing a more complete picture of user preferences. (3) Integrate multi-modal data sources, such as images and videos, to provide additional context and improve user modeling. This could help the model better understand user preferences, even with incomplete textual data. (4) Study the performance of KGCR in real-time systems, focusing on optimizing the model for faster response times and adapting it to handle dynamic user interactions and evolving preferences. (5) Apply KGCR to domains with highly diverse and evolving user preferences, such as news or travel recommendations. This will help validate the model's adaptability and generalizability across different contexts. By addressing these challenges and exploring these research directions, we aim to further improve the KGCR model's performance, robustness, and practical applicability, making it a more effective solution for conversational recommendation tasks in various domains.

Author Contributions: Conceptualization, G.D.; writing—original draft, S.X.; writing—review and editing, Y.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data used in this study are publicly available and include the following: (1) ReDial dataset: This dataset is specifically designed for movie recommendation tasks and contains conversations between users and a recommendation system involving movie recommendations, reviews, and discussions. It can be accessed at (https://github.com/rayliang98/ReDial). (2) MovieLens knowledge graph: This knowledge graph, provided by the GroupLens research group at the University of Minnesota, contains a large number of films, actors, directors, genres, and other entities, as well as their relationships. It can be accessed at (https://grouplens.org/

datasets/movielens/). (3) IMDb dataset: This dataset, used as an external knowledge base for the MovieLens knowledge graph, covers various types of movies and TV series. It can be accessed at (https://www.imdb.com/interfaces/).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Wang, X.; Liu, H. Review of the personalized recommendation system. *Comput. Eng. Appl.* **2012**, *48*, 66–76.
2. Zhao, M.; Huang, X.; Sang, J.; Yu, J. Review of dialogue recommendation algorithm research. *J. Softw.* **2022**, *33*, 4616–4643. [CrossRef]
3. Li, R.; Ebrahimi Kahou, S.; Schulz, H.; Michalski, V.; Charlin, L.; Pal, C. Towards Deep Conversational Recommendations. *Adv. Neural Inf. Process. Syst.* **2018**, *31*, 9748–9758.
4. Li, S.; Xie, R.; Zhu, Y.; Ao, X.; Zhuang, F.; He, Q. User-Centric Conversational Recommendation with Multi-Aspect User Modeling. In Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain, 11–15 July 2022.
5. Lei, W.; He, X.; Miao, Y.; Wu, Q.; Hong, R.; Kan, M.Y.; Chua, T.S. Estimation-Action-Reflection: Towards Deep Interaction Between Conversational and Recommender Systems. In Proceedings of the 13th International Conference on Web Search and Data Mining, Houston, TX, USA, 3–7 February 2020. [CrossRef]
6. Wang, L.; Gao, C.; Huang, C.; Liu, R.; Ma, W.; Vosoughi, S. Embedding Heterogeneous Networks into Hyperbolic Space without Meta-path. *Proc. AAAI Conf. Artif. Intell.* **2021**, *35*, 10147–10155. [CrossRef]
7. Guo, Q.; Zhuang, F.; Qin, C.; Zhu, H.; Xie, X.; Xiong, H.; He, Q. A Survey on Knowledge Graph-Based Recommender Systems. *IEEE Trans. Knowl. Data Eng.* **2022**, *34*, 3549–3568. [CrossRef]
8. Hamilton, W.L.; Ying, Z.; Leskovec, J. Inductive Representation Learning on Large Graphs. *arXiv* **2018**, arXiv:1706.02216.
9. Wang, X.; He, X.; Cao, Y.; Liu, M.; Chua, T.S. KGAT: Knowledge Graph Attention Network for Recommendation. In Proceedings of the KDD '19: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 4–8 August 2019; pp. 950–958. [CrossRef]
10. Gao, C.; Lei, W.; He, X.; de Rijke, M.; Chua, T.S. Advances and Challenges in Conversational Recommender Systems: A Survey. *AI Open* **2021**, *2*, 100–126. [CrossRef]
11. Zhou, K.; Zhao, W.X.; Bian, S.; Zhou, Y.; Wen, J.R.; Yu, J. Improving Conversational Recommender Systems via Knowledge Graph based Semantic Fusion. *arXiv* **2020**, arXiv:2007.04032.
12. Zhu, L.; Huang, X.; Zhao, M.; Sang, J. Multi-round dialogue recommendation system based on negative feedback correction. *J. Comput. Sci.* **2023**, *46*, 1086–1102.
13. Li, X.; Shi, H.; Wang, Y.; Zhang, Y.; Li, X.; Nguyen, C.T. Long Short-Term Planning for Conversational Recommendation Systems. In *Neural Information Processing. ICONIP 2023*; Lecture Notes in Computer Science; Luo, B., Cheng, L., Wu, Z.G., Li, H., Li, C., Eds.; Springer: Singapore, 2024; Volume 14452. [CrossRef]
14. Zhou, K.; Wang, X.; Zhou, Y.; Shang, C.; Cheng, Y.; Zhao, W.X.; Li, Y.; Wen, J.R. CRSLab: An Open-Source Toolkit for Building Conversational Recommender System. *arXiv* **2021**, arXiv:2101.00939.
15. Karthik, R.V.; Ganapathy, S. A fuzzy recommendation system for predicting the customers interests using sentiment analysis and ontology in e-commerce. *Appl. Soft Comput.* **2021**, *108*, 107396. Available online: https://www.sciencedirect.com/science/article/pii/S1568494621003197 (accessed on 27 December 2024). [CrossRef]
16. Aliberti, L.; D'Aniello, G.; Gaeta, M.; Marzolo, A. A Fuzzy Signature-Based Approach for Recommendation Systems. In Proceedings of the 2024 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE), Yokohama, Japan, 30 June–5 July 2024; pp. 1–8. [CrossRef]
17. Rendle, S.; Freudenthaler, C.; Schmidt-Thieme, L. Factorizing personalized Markov chains for next-basket recommendation. In Proceedings of the 19th International Conference on World Wide Web (WWW '10), Raleigh, NC, USA, 26–30 April 2010; Association for Computing Machinery: New York, NY, USA, 2010; pp. 811–820. [CrossRef]
18. Yang, Y.; Jang, H.-J.; Kim, B. A Hybrid Recommender System for Sequential Recommendation: Combining Similarity Models With Markov Chains. *IEEE Access* **2020**, *8*, 190136–190146. [CrossRef]
19. Wang, Y.; Chen, Z.; Zhao, X.; Tan, Z.; Xiao, W.; Cheng, X. Research progress and trends of temporal knowledge graph representation and inference. *J. Softw.* **2024**, *35*, 3923–3951. [CrossRef]
20. He, C.; Wen, B. Recommended model for integrating gating attention mechanisms with bilinear features. *Comput. Appl. Softw.* **2024**, *41*, 291–296.
21. GroupLens. MovieLens 1M Movie Ratings. 1 Million Ratings from 6000 Users on 4000 Movies. Released 2/2003. Permalink. Available online: https://grouplens.org/datasets/movielens/25m (accessed on 20 August 2024).

22. Maas, A.; Daly, R.E.; Pham, P.T.; Huang, D.; Ng, A.Y.; Potts, C. Learning Word Vectors for Sentiment Analysis. In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL 2011), Portland, OR, USA, 19–24 June 2011.
23. Lu, T. Research on the design of customer service intelligent dialogue system based on Rasa framework. *Comput. Telecom* **2022**, *1*, 81–85. [CrossRef]
24. Zhang, M.; Xu, H.; Wang, X.; Zhou, M.; Hong, S. Google TensorFlow Machine Learning Framework and Applications. *Microcomput. Appl.* **2017**, *36*, 58–60. [CrossRef]
25. Fang, Y.; Zhao, X.; Tan, Z.; Yang, S.; Xiao, W. An improved translation-based knowledge graph representation method. *Comput. Res. Dev.* **2018**, *55*, 139–150.
26. Wang, Z. An improved recommendation model of the knowledge graph attention network. *Mod. Comput.* **2022**, *28*, 36–41.
27. Liu, H.; Qu, X.; He, H. A recommended diversity lifting method based on NeuMF. *Comput. Appl. Softw.* **2021**, *38*, 213–220.
28. Sun, J.; Chen, Y. Multi-agent dialogue generation with reinforcement learning. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 31 October–4 November 2018; pp. 2415–2425.
29. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-level control through deep reinforcement learning. *Nature* **2015**, *518*, 529–533. [CrossRef]
30. Yan, X.; Shang, L.; He, X.; Li, X. Graph transformer networks. In Proceedings of the 43rd International Conference on Acoustics, Speech, and Signal Processing (ICASSP), Barcelona, Spain, 4–8 May 2020; pp. 6865–6869.
31. Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Lio, P.; Bengio, Y. Graph attention networks. *arXiv* **2017**, arXiv:1710.10903.
32. Dai, Z.; Chen, E. Transformer-XL: Adaptive receptive field for transformers. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Melbourne, Australia, 15–20 July 2018; pp. 451–460.